

SP25 announcement:
No live lecture on Wed Feb 26 and Mon Mar 3.
Videos will be posted on the website instead.

Lecture 8 (Routing 4)

IP Routers

CS 168, Spring 2025 @ UC Berkeley

Slides credit: Sylvia Ratnasamy, Rob Shakir, Peyrin Kao

IP Routers in Real Life

Lecture 8, CS 168, Spring 2025

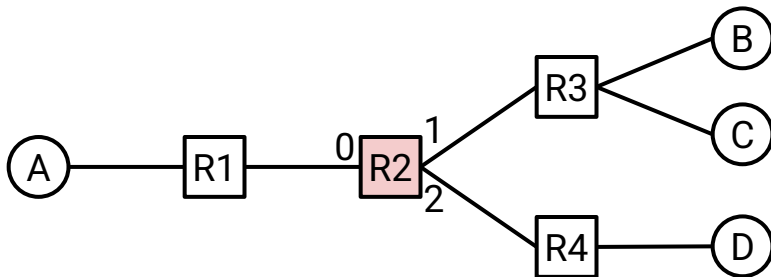
IP Routers

- **Routers in Real Life**
- Router Components (Planes)
- Packet Types
- Forwarding in Hardware
- Efficient Forwarding with Tries

Recall:

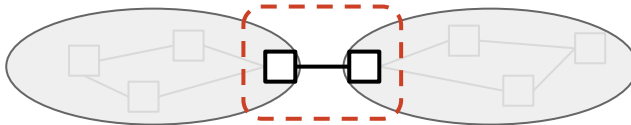
- A router performs routing protocols to learn about routes.
- A router receives packets and forwards them according to the forwarding table.

Today: What do routers actually look like in real life?



R2's Table	
Destination	Port
A	0
B	1
C	1
D	2

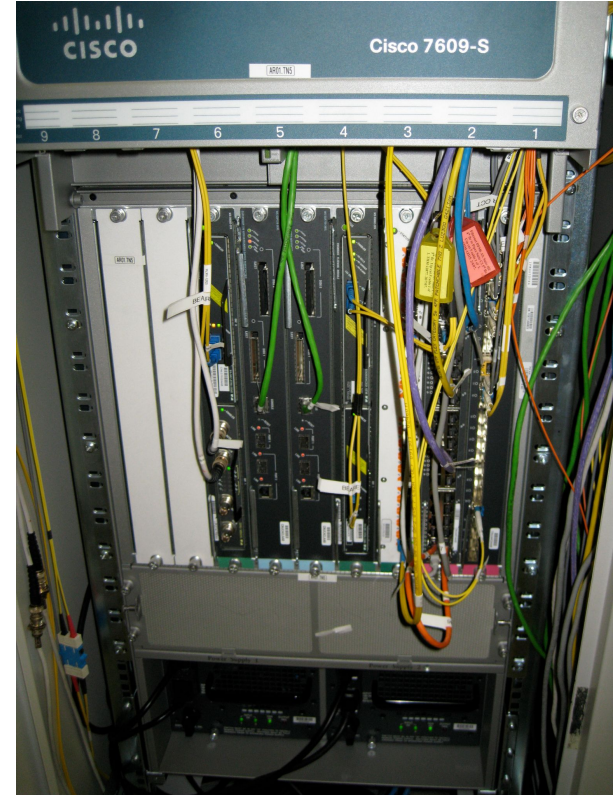
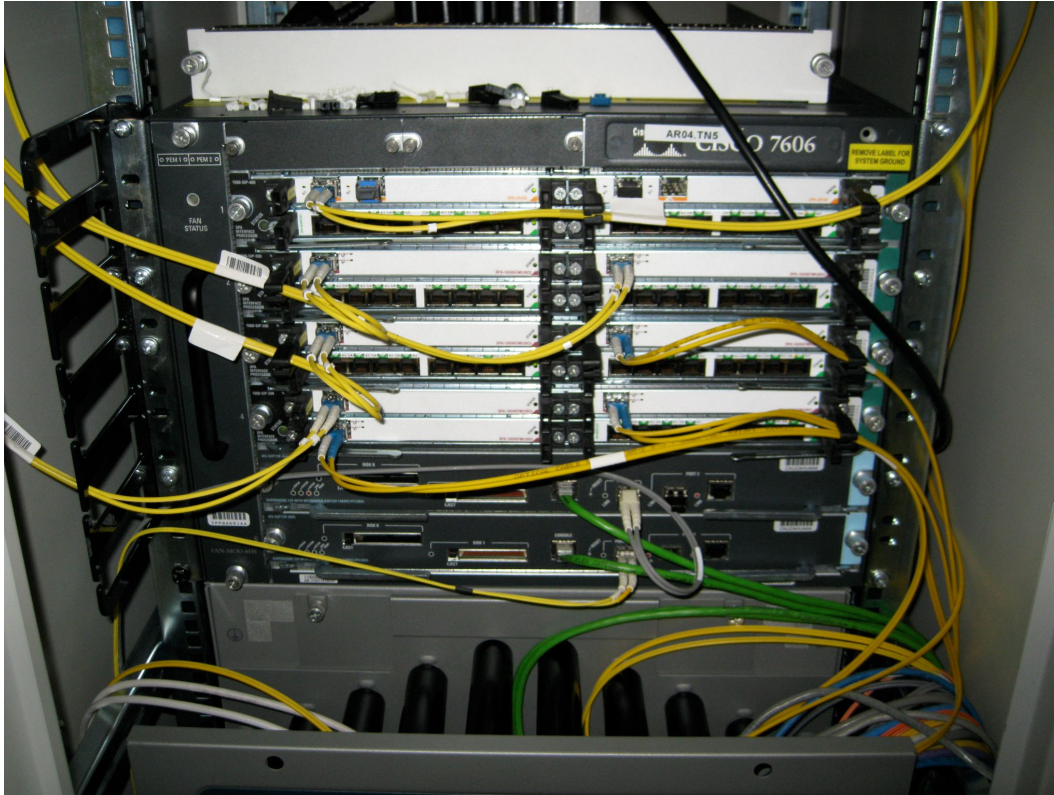
Colocation facility (aka **carrier hotel**): A building where multiple ISPs install routers. Allows for high-performance connections between networks.



In our diagrams, these routers would live in carrier hotels.

Routers in Real Life

A router is just a computer specialized for forwarding packets.



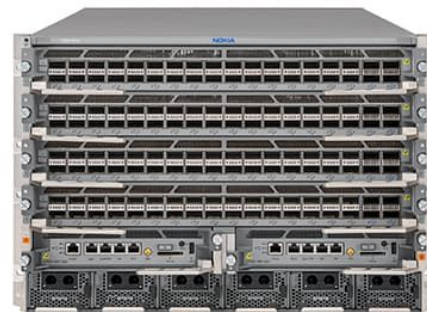
Measuring Router Size

Different dimensions for measuring the size of a router:

- Physical size, number of ports, bandwidth.
- Capacity = number of ports $\times$ speed of each port.
 - The speed of a port is sometimes called its **line rate**.

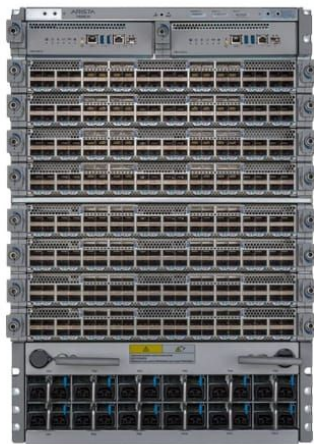
Example:

- A home router might have four 100 Mbps ports, and one 1 Gbps port.
- Total capacity: 1.4 Gbps.



Modern router:

- 288 ports.
- 400 Gbps line rate per port.
- Capacity = 115.2 Tbps.



Next-generation router:

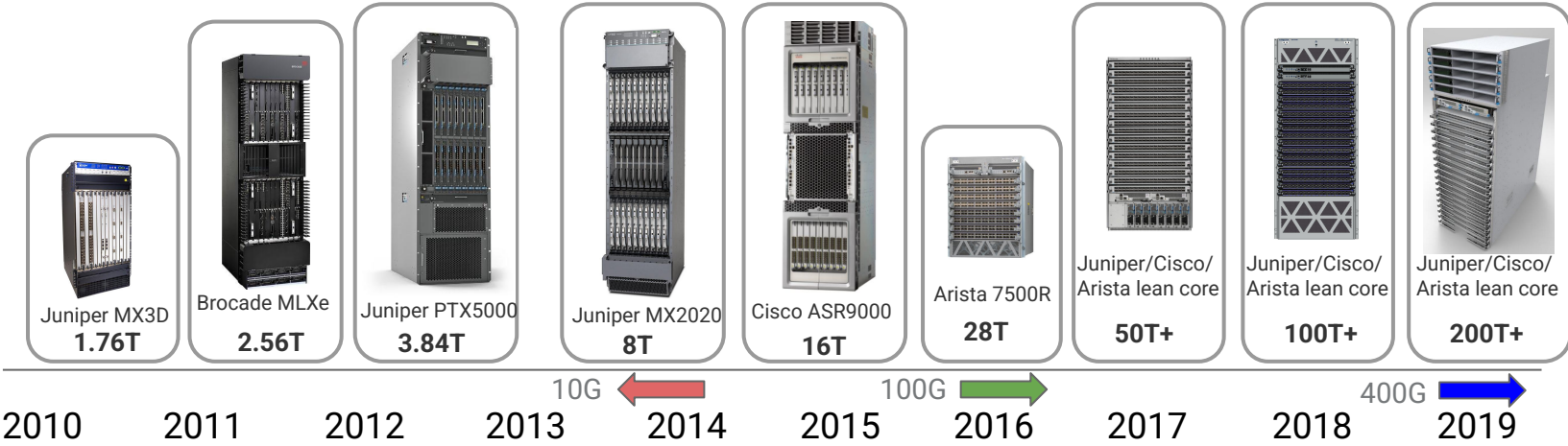
- 288 ports.
- 800 Gbps line rate per port.
- Capacity = 230 Tbps.

Innovation is focused on improving line rate, since we're running out of physical space to add more ports.

Evolution of Router Capacity

Modern routers are facing physical constraints: size, power, cooling, etc.

Rate of growth is slowing: 10G → 100G → 400G → 800G.



Router Components (Planes)

Lecture 8, CS 168, Spring 2025

IP Routers

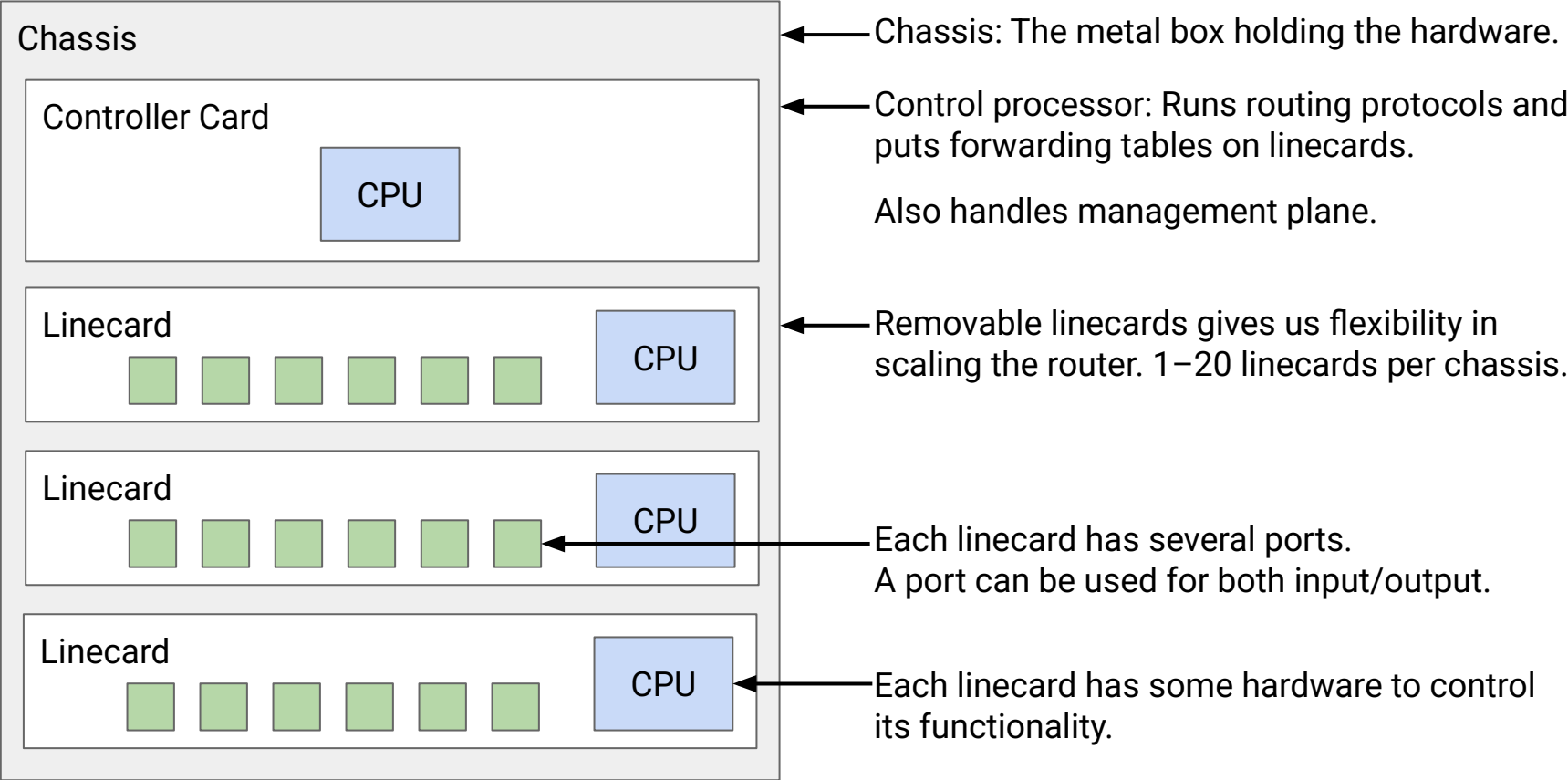
- Routers in Real Life
- **Router Components (Planes)**
- Packet Types
- Forwarding in Hardware
- Efficient Forwarding with Tries

The components of a router can be split into 3 planes:

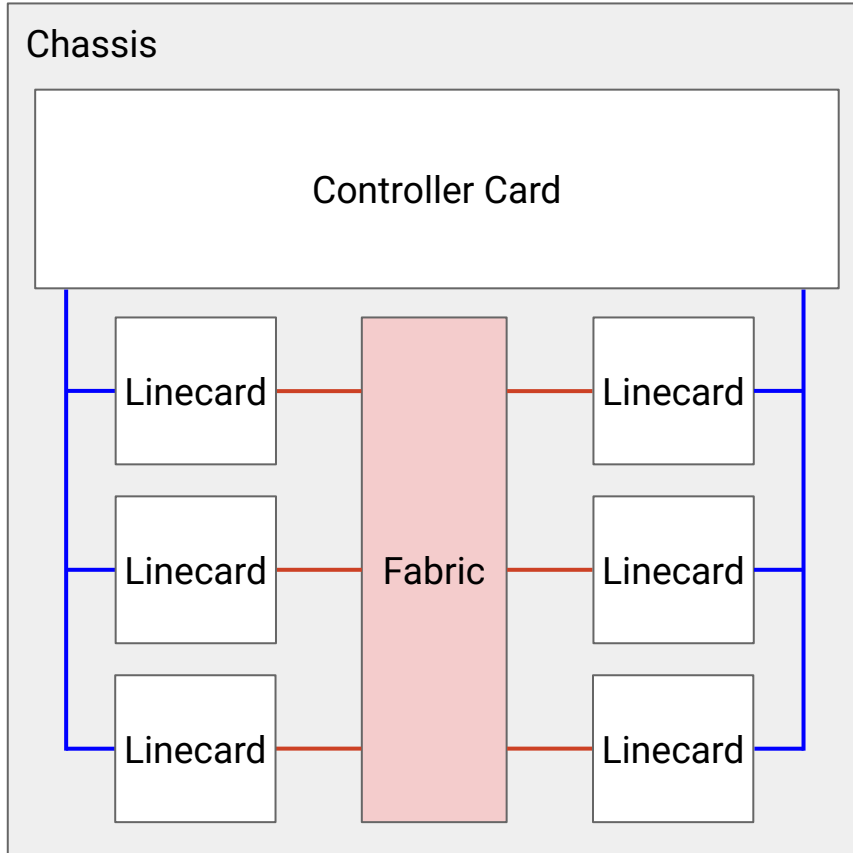
- The **data plane** handles forwarding packets.
 - Used every time a packet arrives: Nanosecond time scale.
 - Operates locally. No coordinating with other routers.
- The **control plane** performs routing protocols.
 - Used every time the network topology changes. Second time scale.
- The **management plane** lets the operator interact with the router.
 - Operator uses **network management system** (NMS) to interact with router.
 - Tell the router what to do, and monitor what the router is doing.
 - Time scale of seconds/minutes.
 - Example: Assigning costs to links.
 - Example: How much traffic is being sent on each link?

Each plane is optimized for its tasks and its time scale.

What's Inside a Router? – View of Components



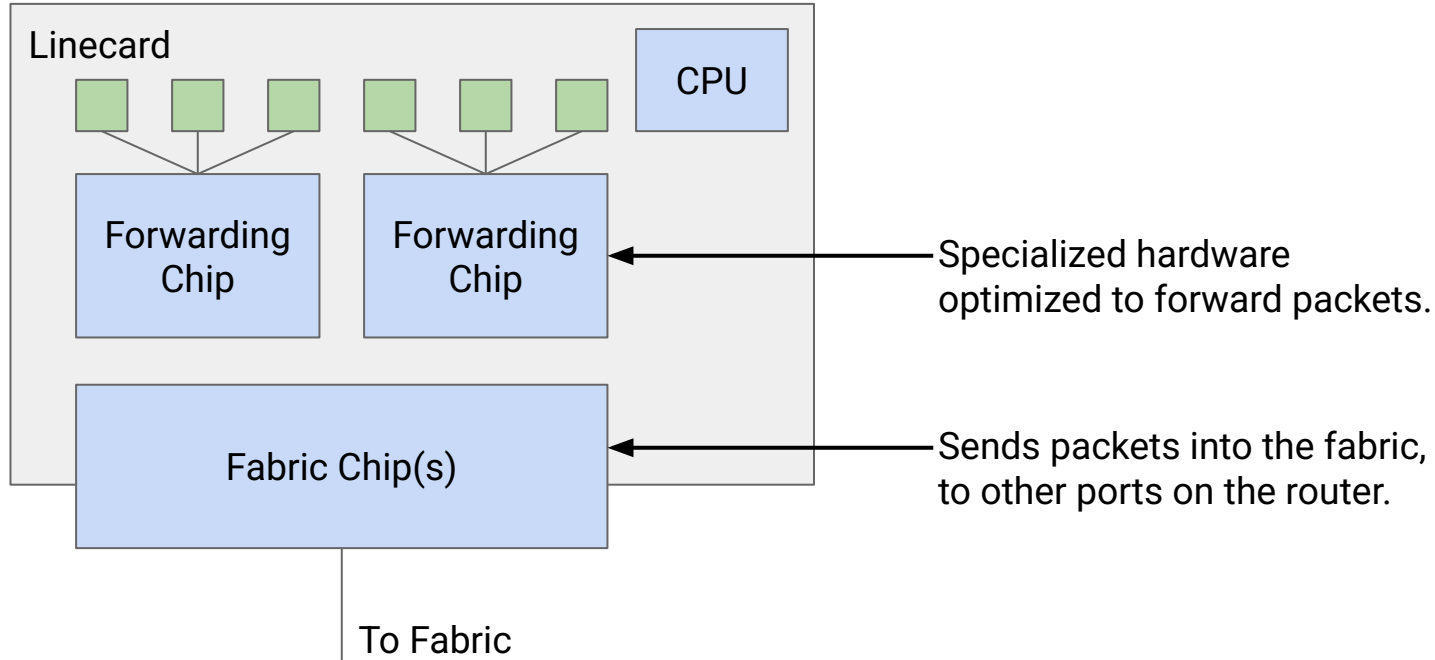
What's Inside a Router? – View of Links



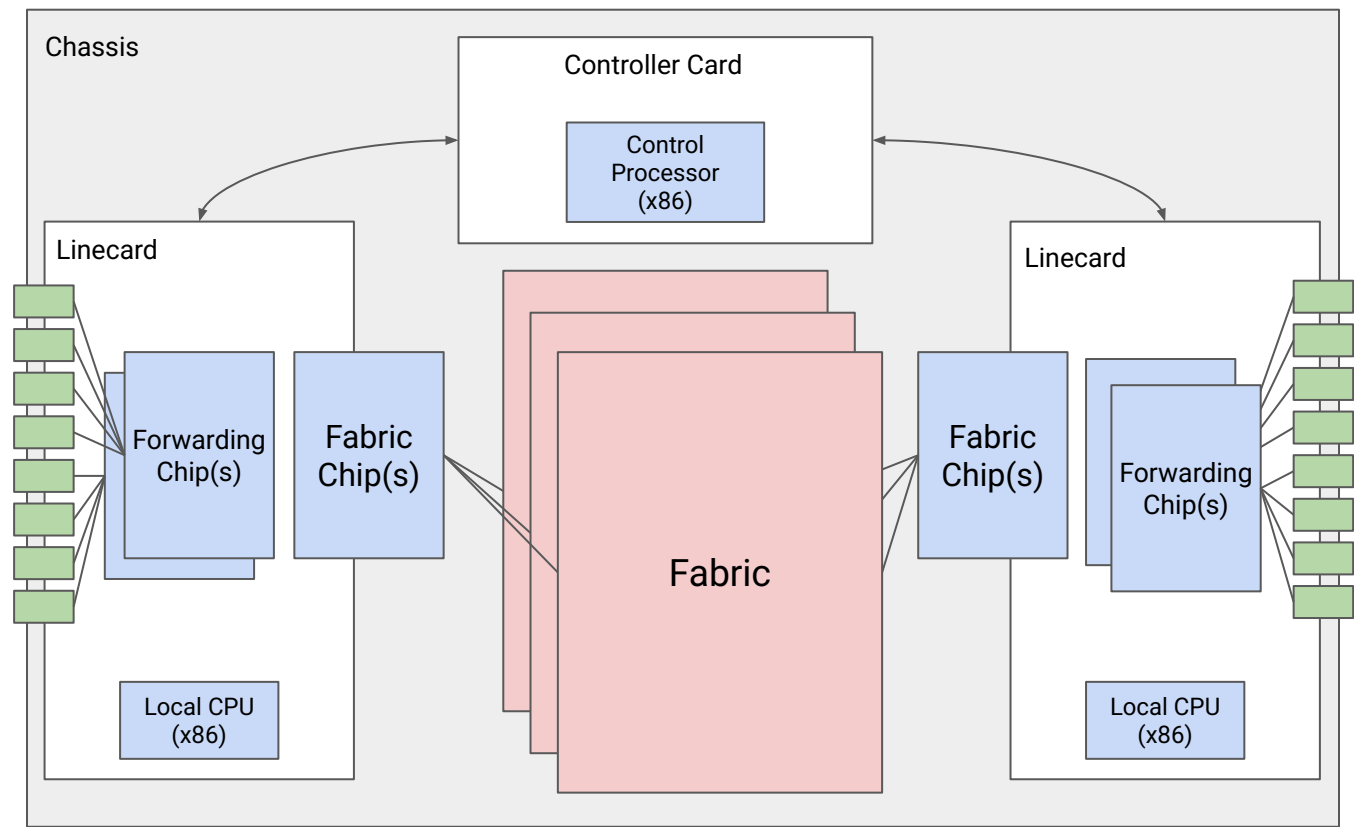
Controller card is connected to the linecards.

Each linecard is connected to the **fabric**:
A bunch of wires providing
high-bandwidth, fault-tolerant
interconnection between linecards.

What's Inside a Router? – View of a Linecard

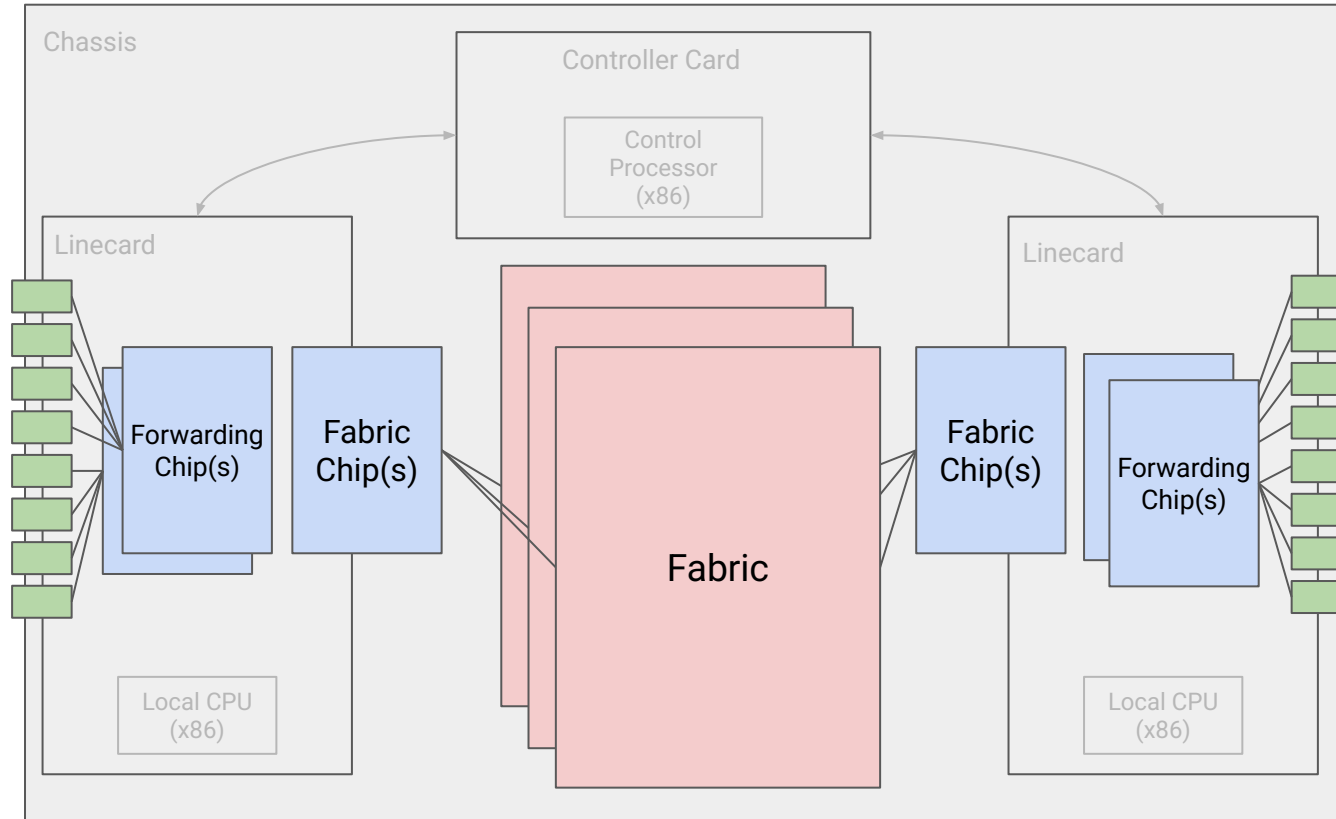


What's Inside a Router? – Full View



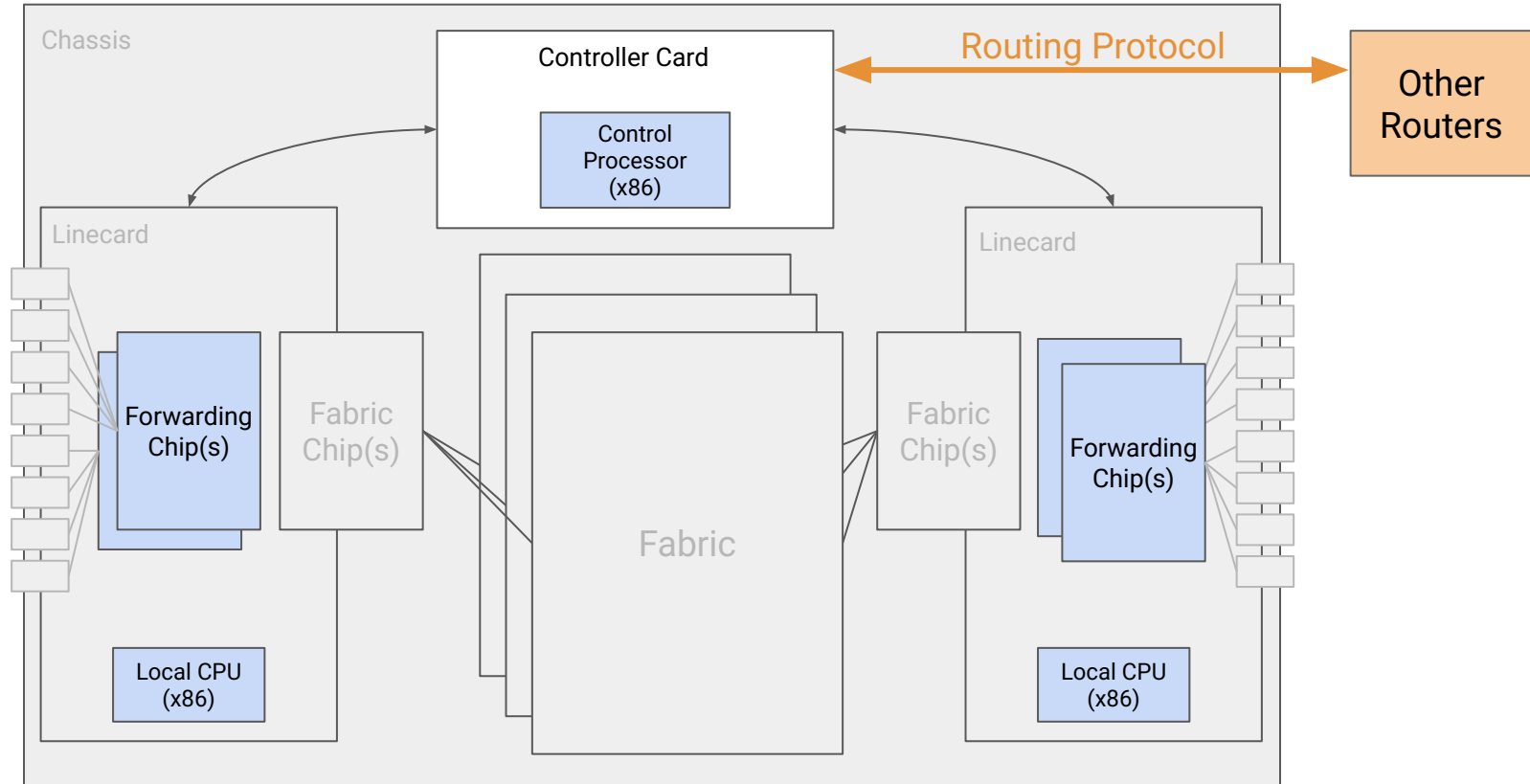
What's Inside a Router? – Data Plane

Data: Packet travels from port to port, via forwarding chips and fabric.



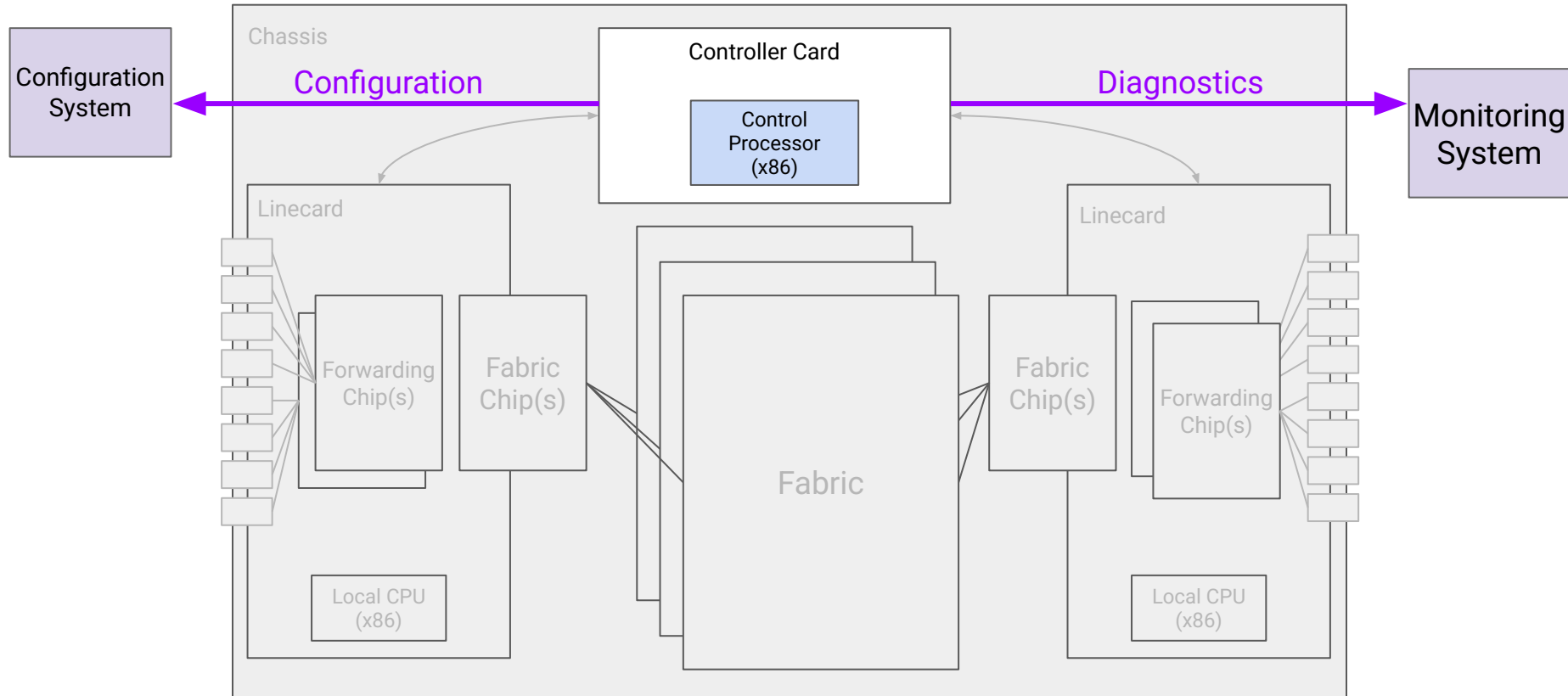
What's Inside a Router? – Control Plane

Control: Controller card talks with other routers, and programs linecards with routes.



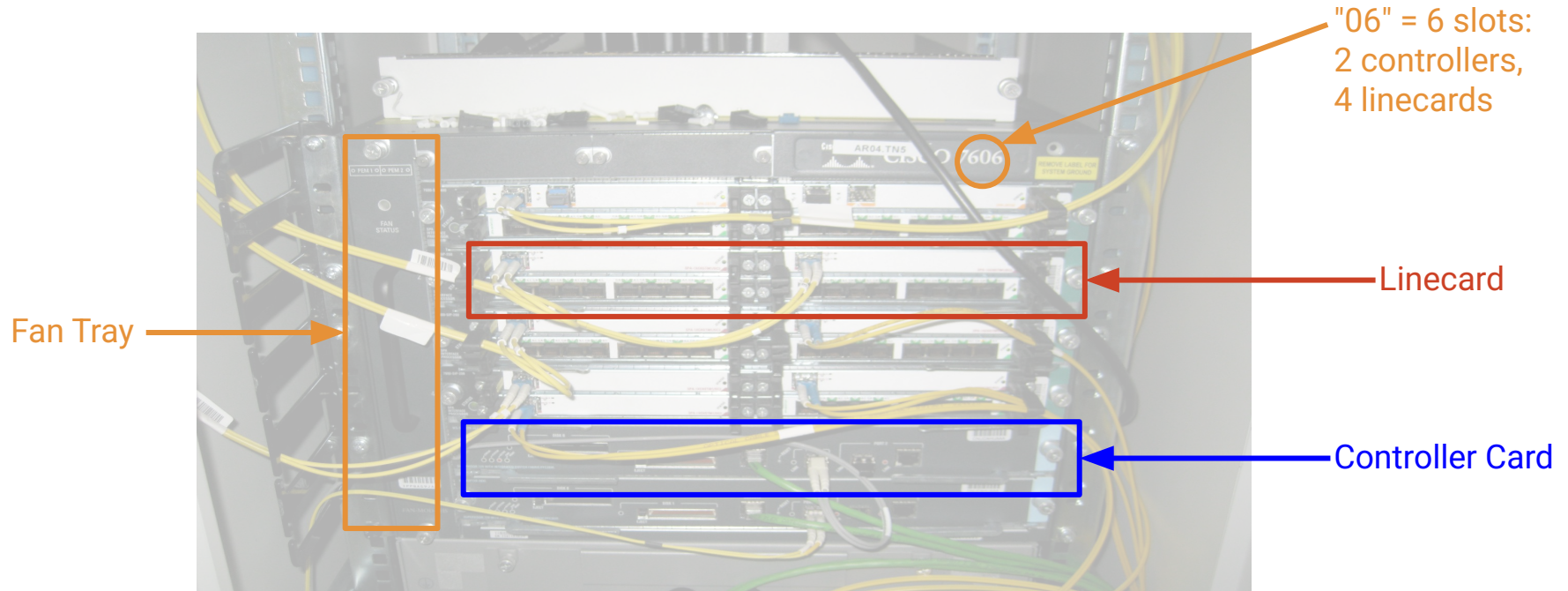
What's Inside a Router? – Management Plane

Management: Controller card talks with operator.



What's in a Router?

A router is really a *cluster* of computers specialized for forwarding packets.



Packet Types

Lecture 8, CS 168, Spring 2025

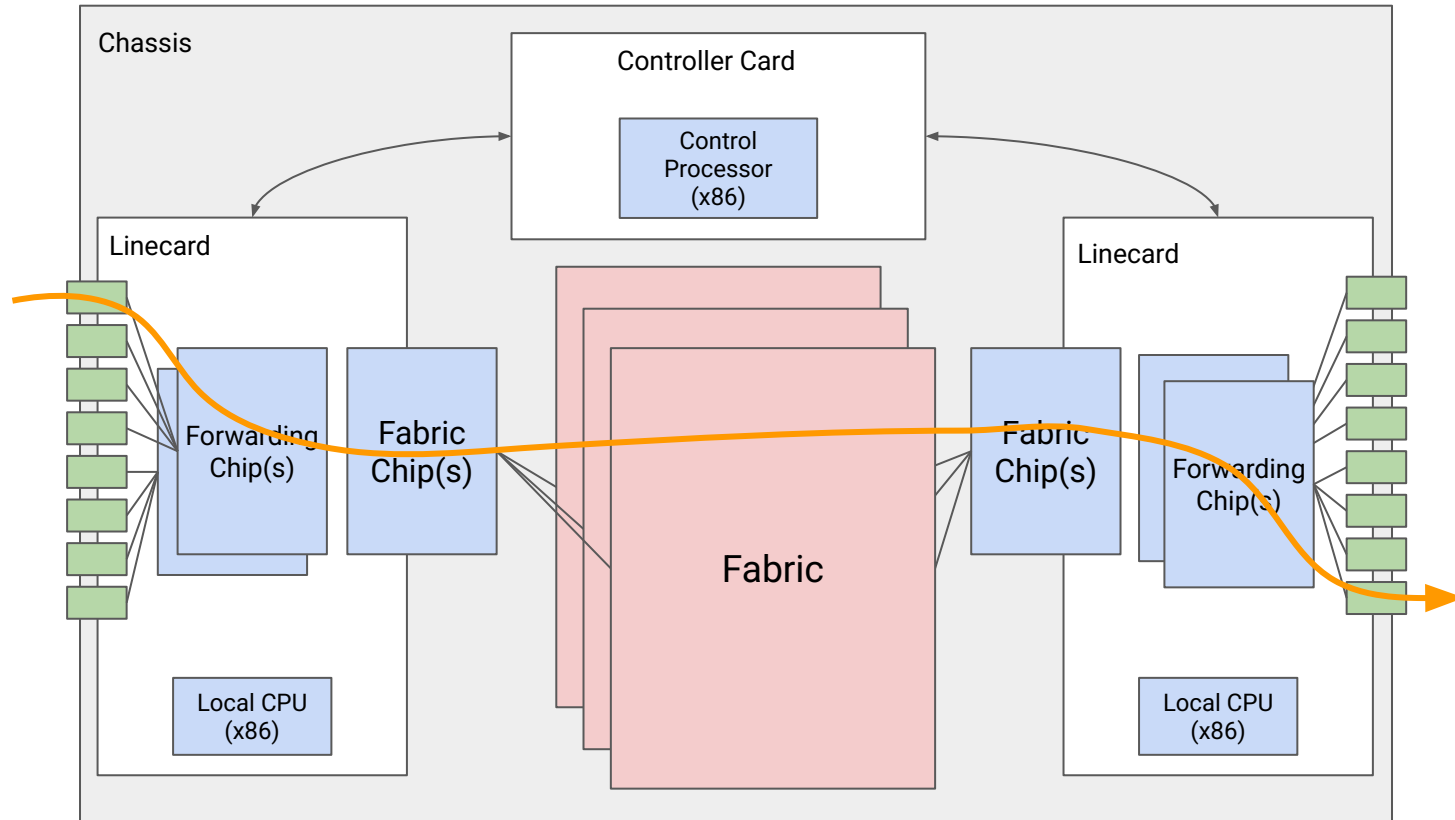
IP Routers

- Routers in Real Life
- Router Components (Planes)
- **Packet Types**
- Forwarding in Hardware
- Efficient Forwarding with Tries

3 types of packets can arrive at a router.

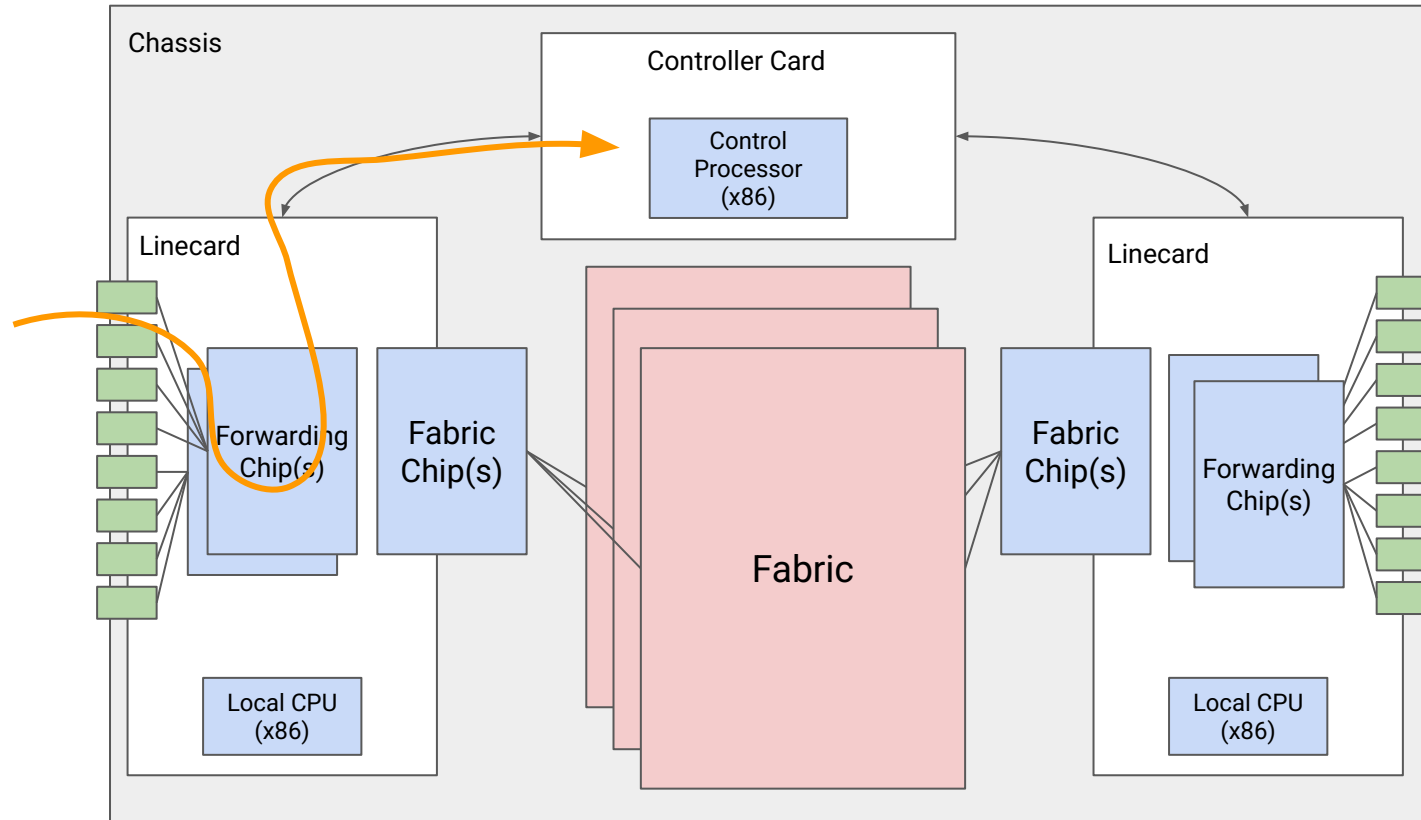
- **User packet:** The router needs to forward this packet toward its destination.
 - Most common type of packet.
 - Forwarding chip looks up which port to forward this packet.
 - If outgoing port is on a different linecard, send packet through fabric to that linecard.
- **Control plane traffic:** Packets intended for the router itself.
 - Example: Advertisements in routing protocols.
 - Forwarding chip sends the packet to the controller for processing.
- **Punt traffic:** Packets intended for the user, but requiring extra processing.
 - Example: Packet TTL has expired. Need to send back an error message.
 - Forwarding chip "punts" the packet to the controller for processing.

User packet: Forward according to installed routes.



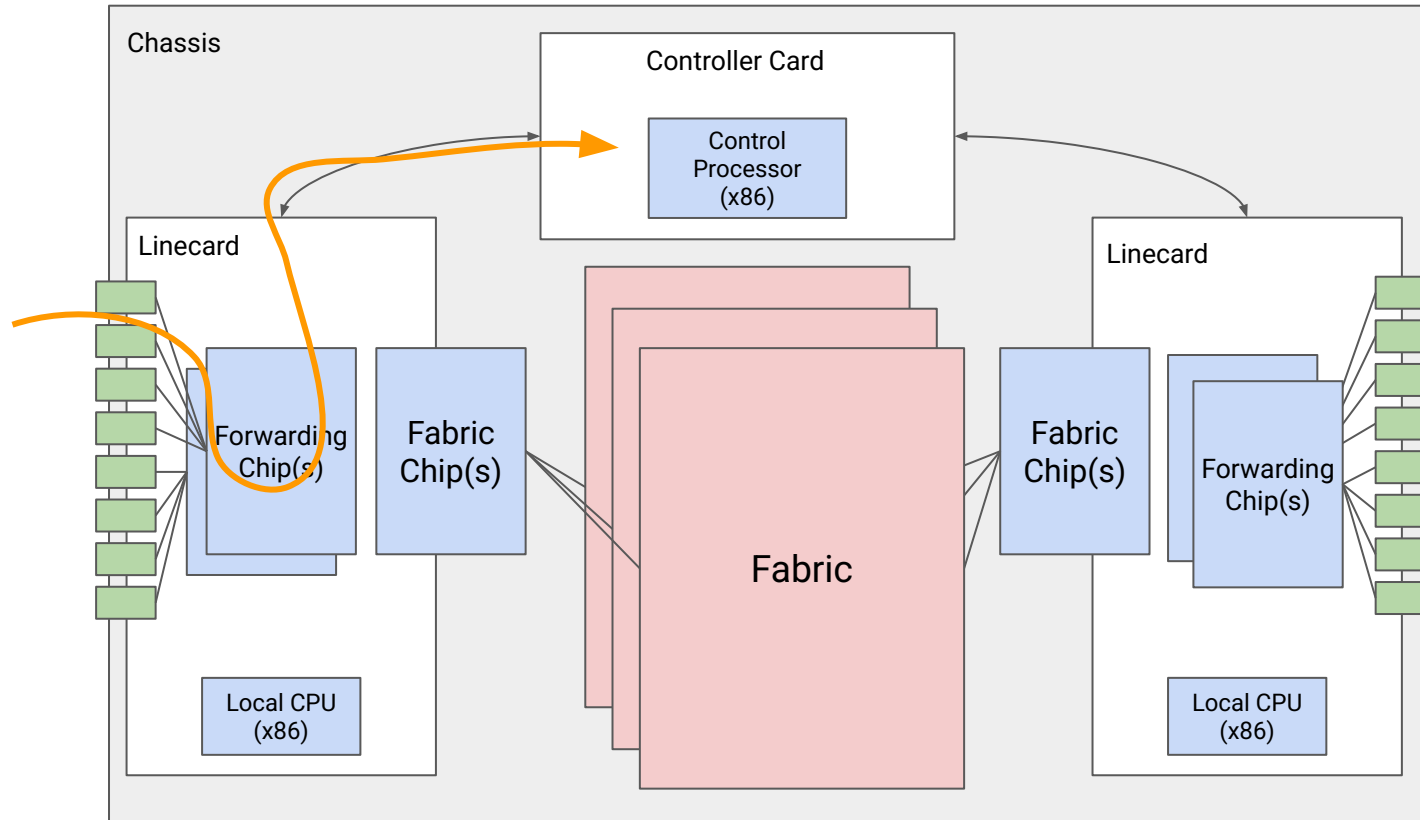
Types of Packets – Control Plane Traffic

Control plane traffic: Packets destined for this router (e.g. routing advertisement).



Types of Packets – Punt Traffic

Punt traffic: User packets that need extra processing (e.g. TTL expired).



Why build routers like this? Why not just use a general-purpose CPU?

Reason: *Scale*.

- Assuming 64-byte packets and 400 Gbps, we have to process 781 million packets, per second, per port.
- Across 36 ports, the router has to process 56 billion packets per second.

We can't achieve this scale in software.

- You could write software to forward one packet per 10ms = 0.00001s.
- We need to forward one packet per 10ns = 0.00000001s.

Router functionality must be implemented directly on hardware.

- User packets take the "fast path" in hardware.
- Only use the "slow path" in software when necessary.

Forwarding in Hardware

Lecture 8, CS 168, Spring 2025

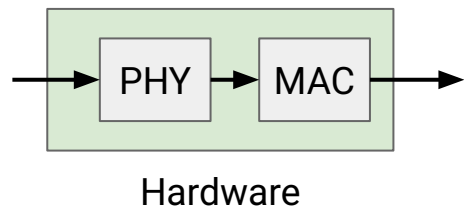
IP Routers

- Routers in Real Life
- Router Components (Planes)
- Packet Types
- **Forwarding in Hardware**
- Efficient Forwarding with Tries

When a packet arrives, what does the input linecard need to do?

1. Receive the packet from other systems.

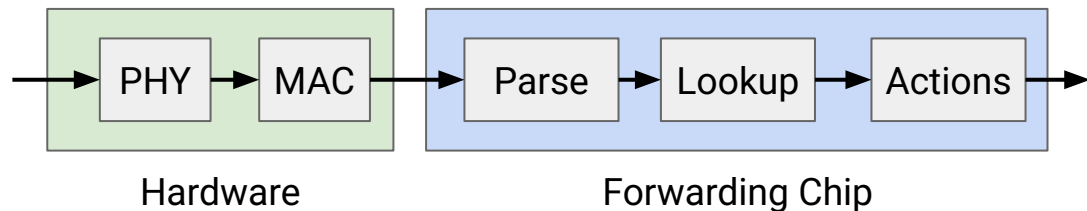
- PHY (physical layer): Decode the optical/electrical signal into 1s and 0s.
- MAC (link layer): Perform link-layer operations.
- These are implemented in hardware.



When a packet arrives, what does the input linecard need to do?

2. Process the packet.

- Parse the packet to understand its headers, e.g. IPv4 or IPv6.
- Look up the next hop in the forwarding table.
- Update the packet.
 - Decrement TTL, update checksum, fragment packet if it's too big, etc.

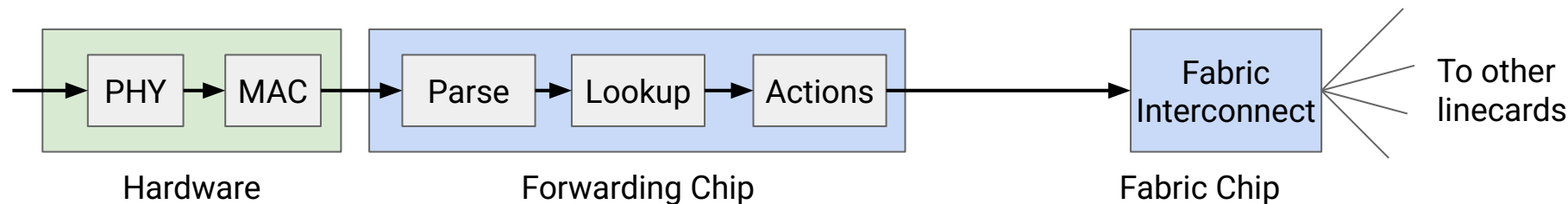


Forwarding Pipeline (3/3)

When a packet arrives, what does the input linecard need to do?

3. Send the packet onwards.

- Fabric interconnect chip sends packets to other linkcards via inter-chassis links.

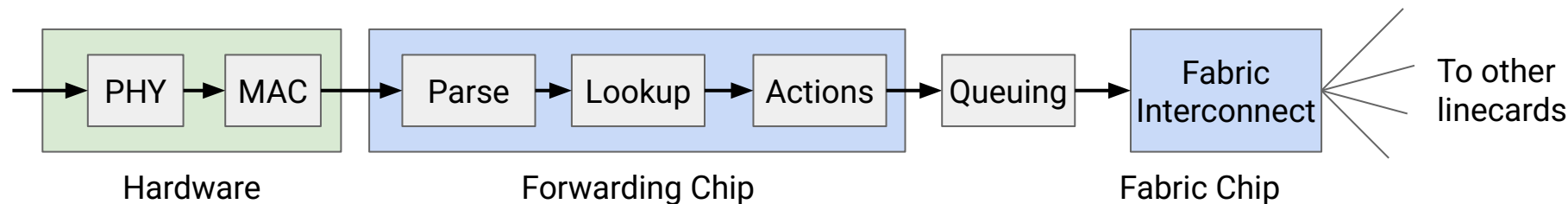


Forwarding Pipeline – Queuing

Many possible queuing approaches: We'll assume the simplest one.

- Classification: Which queue should this packet be put into?
 - One queue per input port? One queue per flow?
 - Assume no classification.
- Buffer management: Should we drop packets?
 - Assume tail drop: Drop packet if queue is full.
- Scheduling: What order do we send out packets?
 - Assume FIFO: Send packets in the order they arrive.

Alternate complex approaches can be used to implement business objectives.



Main challenge: *Speed*.

- We have to forward one packet every few nanoseconds.
- One chip is handling packets from many ports.
- We have to do multiple operations to forward a packet.

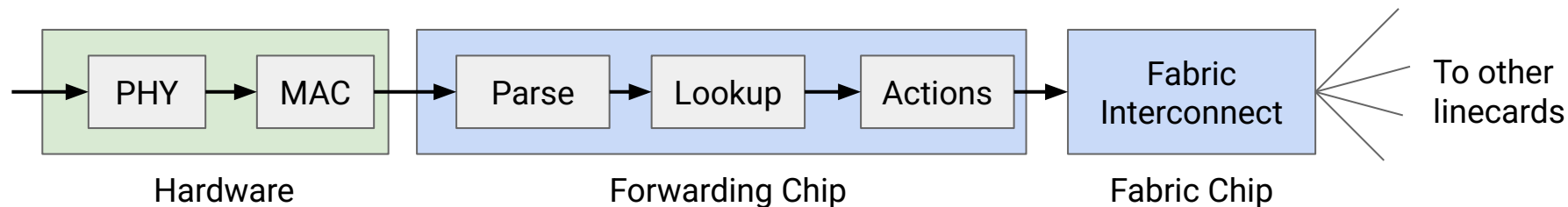
Network processors are specialized to perform forwarding quickly.

- Trade-off: The chip has limited functions. Can't run an arbitrary C program on it.
- Any special processing can be punted to the controller.

Scaling Forwarding Hardware

How hard is it to implement operations in hardware?

- Parse: Easy. Read specific bits of the packet.
- Lookup: ??? ← Doing efficient lookup is our challenge!
- Actions:
 - Some are easy: Update checksum, decrement TTL.
 - Some are harder: Special options, fragment packet if it's too big.
 - The Internet avoids using the harder ones. They usually require punting.
- Fabric interconnect: Dedicated chip with limited features ensures speed.



Efficient Forwarding with Tries

Lecture 8, CS 168, Spring 2025

IP Routers

- Routers in Real Life
- Router Components (Planes)
- Packet Types
- Forwarding in Hardware
- **Efficient Forwarding with Tries**

Efficient Forwarding

The forwarding table is a map (key-value pairs).

How do we do fast lookups?

Challenges:

- Entries can contain a range of addresses, not just one.
- Ranges might overlap. A destination can match multiple entries.

Naive solution: Write out the whole range.

- Table gets really big.
- If a route changes, we have to update tons of entries.
- We need something smarter.

R2's Table	
Destination	Port
2.1.1.0/24	5



R2's Table	
Destination	Port
2.1.1.0	5
2.1.1.1	5
2.1.1.2	5
2.1.1.3	5
2.1.1.4	5
...	...
2.1.1.252	5
2.1.1.253	5
2.1.1.254	5
2.1.1.255	5

Longest Prefix Match

We want a fast implementation of **longest prefix match**.

- If the address matches multiple prefixes, take the most specific (longest) match.
- If the address matches no prefixes, take the default route.
- If there's no default route, drop the packet.

These two prefixes match.
The first one is longer.

R2's Table				
Destination				Port
11101000	01100101	111.....		5
11101000	01100...			9
11101100	01100101	111.....		7
11111...				2

Destination: 11101000 01100101 11101011 11000110

Longest Prefix Match

Naive longest prefix match implementation:

- For each prefix, check if it fully matches. If yes, add to list.
- Pick the longest prefix in the list.
- If list is empty, use default route (or drop).

Requires scanning every entry. $O(N)$ runtime for table with N entries.

These two prefixes match.
The first one is longer.

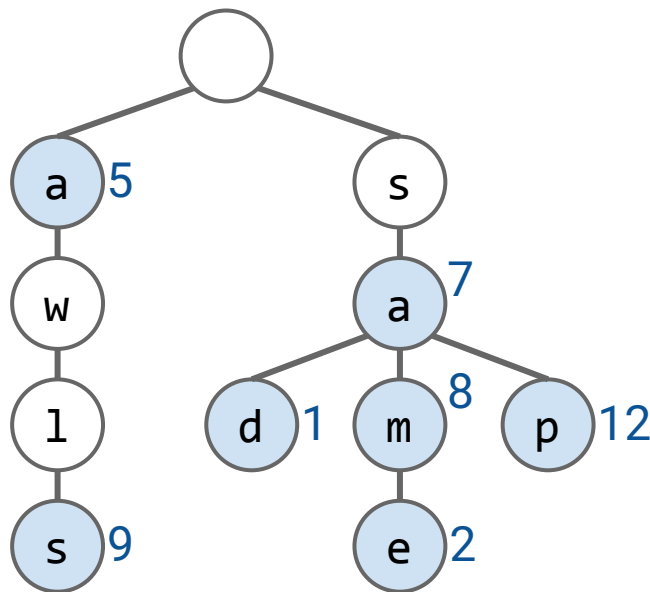
R2's Table				
Destination				Port
11101000	01100101	111.....		5
11101000	01100...			9
11101100	01100101	111.....		7
11111...				2

Destination: 11101000 01100101 11101011 11000110

Is there a map data structure that supports efficient longest prefix match?

- We can use a more obscure data structure: **Tries**. ← Maybe familiar if you've taken UC Berkeley CS 61B.
- Idea: Spell out each key, one letter/digit at a time.
- A node is marked blue if walking from root to that node forms a valid key.

Key	Value
a	5
awls	9
sa	7
sad	1
sam	8
same	2
sap	12

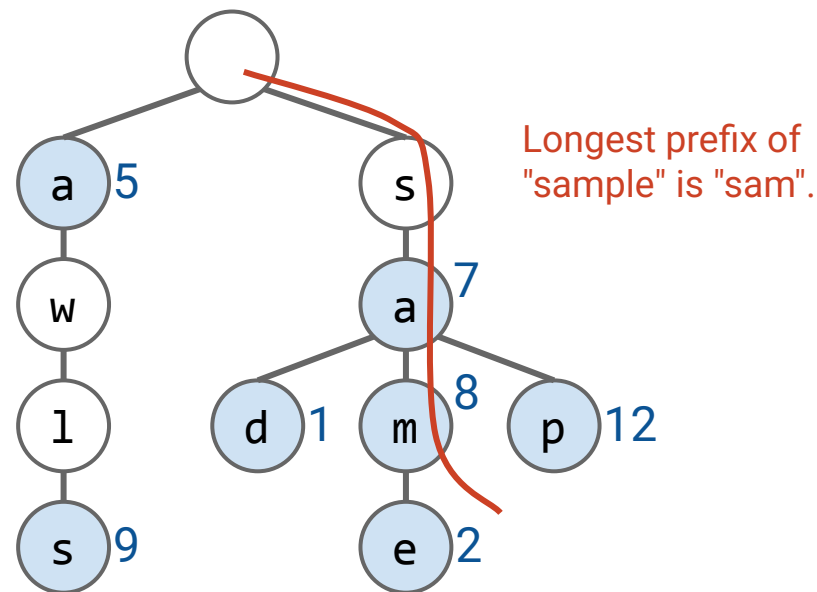


Tries – Conceptual

Longest prefix matching on a trie:

- Start at the root, and spell out the word.
- Remember most recent (longest) key (blue node) you see along the way.
- Stop when you're done, or fall off the tree. Return the longest key you saw.

Key	Value
a	5
awls	9
sa	7
sad	1
sam	8
same	2
sap	12



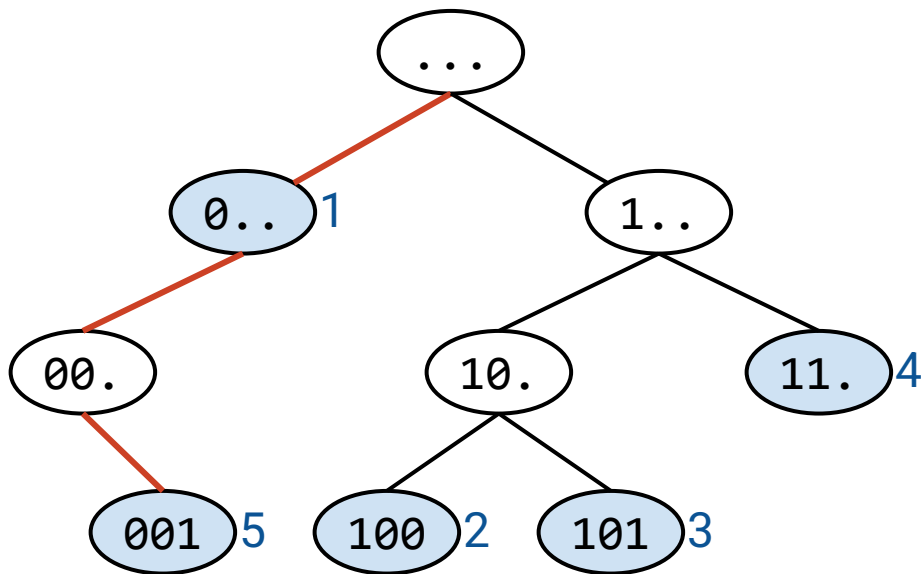
Efficient IP Lookup with Tries

Longest prefix matching on a trie:

- Start at the root, and spell out the word.
- Remember most recent (longest) key you see along the way.
- Stop when you're done, or fall off the tree. Return the longest key you saw.

Key	Value
0..	1
100	2
101	3
11.	4
001	5

Longest prefix of
00100 is 001.



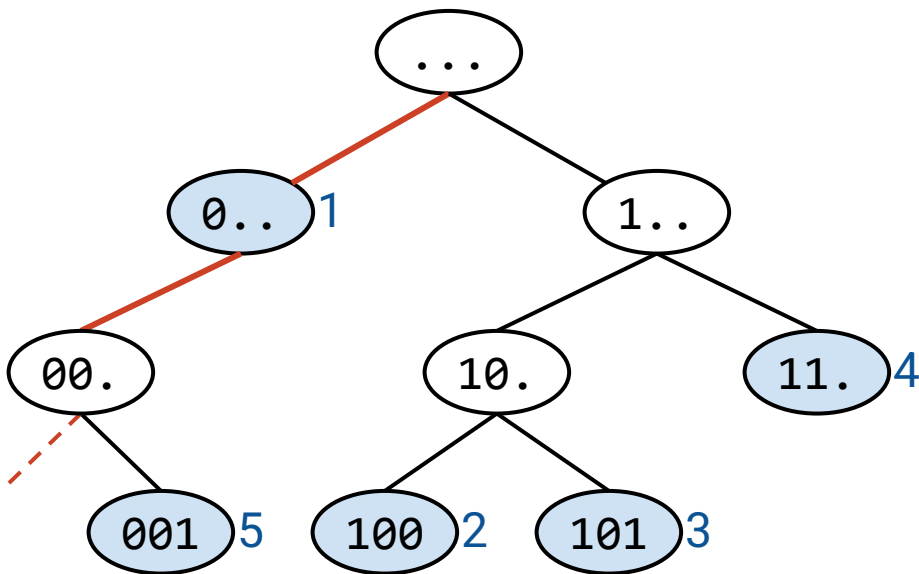
Efficient IP Lookup with Tries

Longest prefix matching on a trie:

- Start at the root, and spell out the word.
- Remember most recent (longest) key you see along the way.
- Stop when you're done, or fall off the tree. Return the longest key you saw.

Key	Value
0..	1
100	2
101	3
11.	4
001	5

Longest prefix of
00000 is 0.



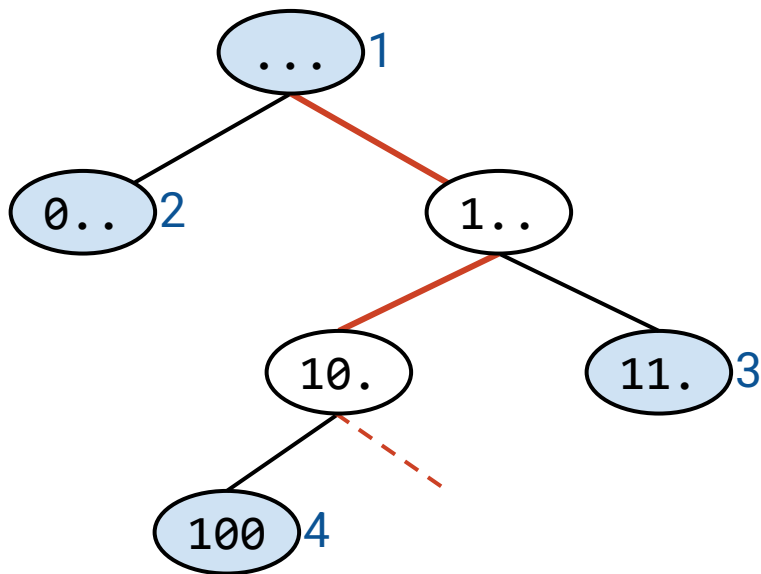
Efficient IP Lookup with Tries

Longest prefix matching also works with default route.

- If you see another prefix, it will be returned over the default route.
- If you finish spelling or fall off the tree without seeing any other prefix, the default is returned.

Key	Value
...	1
0..	2
11.	3
100	4

Longest prefix of
10100 is default.



Runtime of longest prefix matching in a trie is *constant*: $O(1)$.

- We'll only visit at most 32 nodes from spelling out the IP address.

All routers have efficient longest prefix matching functionality.

Some use more complex solutions with heuristics and optimizations.

- Some destinations are more popular than others.
- Some ports have more destinations.
- Longest prefix to external networks is /24 (e.g. you won't see a 29-bit prefix).
- We could optimize for fast trie/table updates too.

Routers have different planes.

- Data plane: Forward packets.
- Control plane: Programming forwarding entries and handling exception packets.
- Management plane: Configure and monitor router functionality.

Data plane leverages tradeoffs in software vs. hardware packet processing.

- Software: Flexible but slow.
- Hardware: Inflexible but fast.

Data plane challenges: Speed!

- Some operations are easy, e.g. update packet header.
- Longest-prefix lookup on destination address is harder. Used tries for efficiency.